

CAPSTONE PROJECT REPORT

WIPRO-NGA Training - Great Learning

Healthcare Appointment & Care Analytics Platform

A secure, layered .NET platform for healthcare appointment management, patient portal access, REST APIs, and care analytics.

Submitted by
Rithin Vishwas

Program: Full-Stack .NET Development Capstone
Submission Date: 06 June 2026

Table of Contents

1. Executive Summary
2. Project Overview
3. System Architecture
4. Data Flow Diagrams
5. Database Design
6. File Structure
7. Core Business Rules
8. API Documentation
9. Security Implementation
10. Frontend / Backend Connectivity
11. Report Generation Process
12. User Interface Screenshots and Outputs
13. DevOps and AI Modernisation
14. Testing and Validation
15. Final Demonstration
16. Challenges and Learnings
17. Future Enhancements
18. Ideal Scenario and Assumptions
19. Comparison with Live Healthcare Applications
20. Conclusion
21. References

1. Executive Summary

The Healthcare Appointment & Care Analytics Platform is a layered ASP.NET Core solution built for a healthcare provider to manage patients, doctors, appointments, role-based access, and analytics reporting. The solution includes a server-rendered MVC portal for administrators and patient users, a JWT-secured Web API for Swagger/Postman/mobile clients, and a Microsoft SQL Server database with normalized tables, views, stored procedures, triggers, and EF Core mapping.

Business Objective

The business objective is to improve care delivery by reducing manual appointment coordination, preventing appointment conflicts, enabling secure patient self-service, and giving hospital staff a reliable dashboard for appointments and care analytics.

Project Scope

- Admin login, dashboard, patient management, appointment management, and analytics reports.
- User registration, patient profile creation, appointment booking, and personal appointment history.
- JWT-secured REST API with Swagger testing for Auth, Patients, Doctors, Appointments, and Analytics.
- SQL Server schema with normalized tables, audit triggers, reporting views, stored procedures, and seed data.
- SOLID-based layered architecture with Core, Infrastructure, API, MVC, and Tests projects.
- Docker, GitHub Actions, Azure conceptual deployment, and GitHub Copilot prompt log.

Key Features

- Cookie authentication for MVC portal and JWT authentication for REST API.
- Role-based authorization for Admin, User, and clinical operations.
- Appointment conflict validation for doctor and patient schedules.
- ADO.NET report generation using stored procedures and SQL views.
- Database audit logging for appointment create/update activity.
- Security controls for SQL injection, CSRF, input validation, and security headers.

2. Project Overview

2.1 Problem Statement

Healthcare providers often manage patient appointments through fragmented channels such as phone calls, spreadsheets, manual registers, and ad hoc communication. This creates duplicate bookings, poor visibility of appointment status, limited auditability, and difficulty in generating care analytics. The project solves this by providing a single integrated platform for patient records, doctor selection, appointment booking, secure API access, and reporting.

2.2 Business Context

The platform supports two primary personas: the Administrator / Hospital Staff, who manages patients and appointment operations, and the Registered Patient User, who can register, create a patient profile, and book appointments. The Web API supports future mobile clients and external healthcare integrations.

2.3 Technology Stack

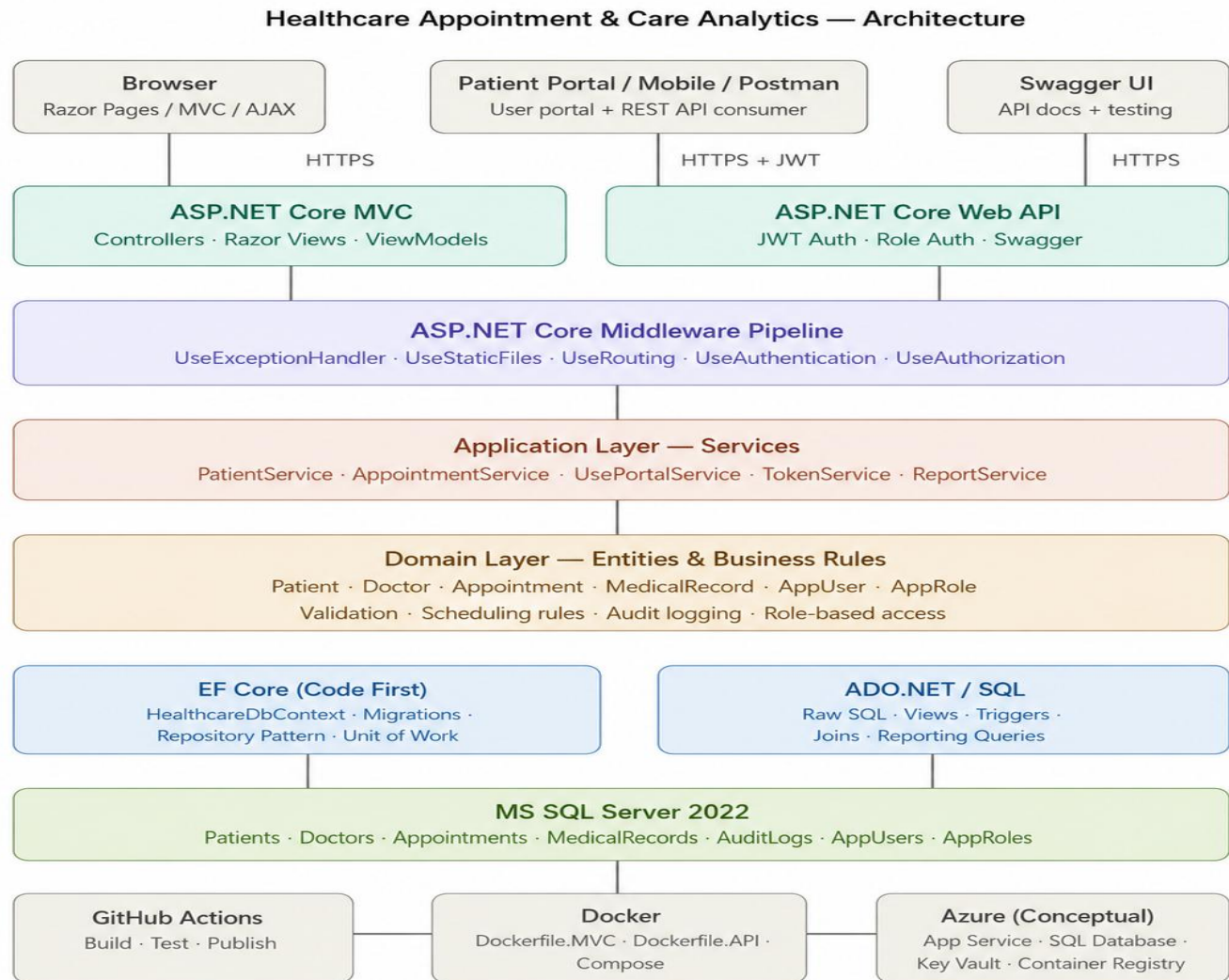
Layer	Technology
Backend API	.NET 8, ASP.NET Core Web API, C#
Frontend Portal	ASP.NET Core MVC, Razor Views, ViewModels, AJAX, Bootstrap-style UI
Core Layer	Entities, DTOs, enums, exceptions, interfaces, domain services
Infrastructure	EF Core, SQL Server provider, repositories, Unit of Work, services
Database	Microsoft SQL Server 2022 / SQL Server Express
Authentication	MVC cookies, JWT Bearer tokens, PBKDF2 password hashing
Validation	DataAnnotations, ModelState, domain service validation, SQL constraints
Documentation	Swagger / OpenAPI, Postman collection, diagrams, final report
Testing	xUnit tests for appointment business rules
DevOps	Dockerfiles, docker-compose, GitHub Actions, Azure conceptual model
AI Tooling	GitHub Copilot usage log and conceptual AI/MCP extension

3. System Architecture

3.1 Overview

The solution follows a clean layered architecture. The MVC project handles browser UI and Razor views. The API project exposes REST endpoints secured with JWT. The Core project owns domain entities, DTOs, business interfaces, and business rules. The Infrastructure project implements EF Core persistence, repositories, Unit of Work, password hashing, JWT token generation, and ADO.NET report generation. SQL Server stores operational and reporting data.

3.2 High-Level Architecture



3.3 Architecture Explanation

Browser requests reach ASP.NET Core MVC controllers, which validate model state and delegate work to services and repositories. API requests pass through JWT Bearer middleware before reaching protected controllers. Both MVC and API paths share Core interfaces and Infrastructure implementations. Database communication is handled through EF Core for CRUD operations and ADO.NET for reporting. Cross-cutting concerns such as exception handling, security headers, authentication, authorization, and static file handling are configured in Program.cs and middleware.

4. Data Flow Diagrams

4.1 DFD Level 0 - Context Diagram

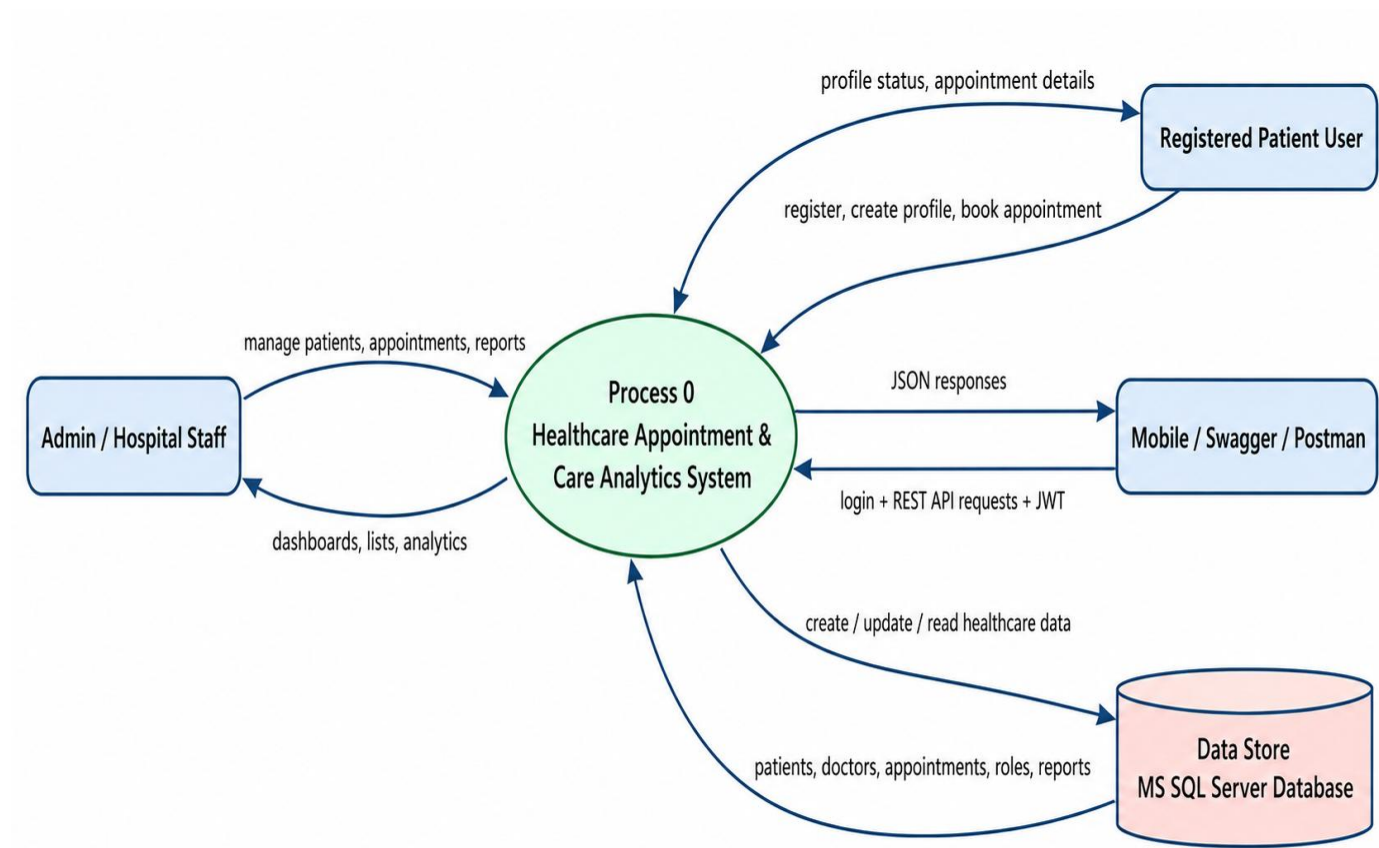


Figure 2: DFD Level 0 - Context Diagram

The context diagram shows the system boundary and its interaction with external actors: Admin / Hospital Staff, Registered Patient User, Mobile / Swagger / Postman clients, and the SQL Server database.

4.2 DFD Level 1 - Internal Process Flow

DFD Level 1 — Main Processes and Data Stores

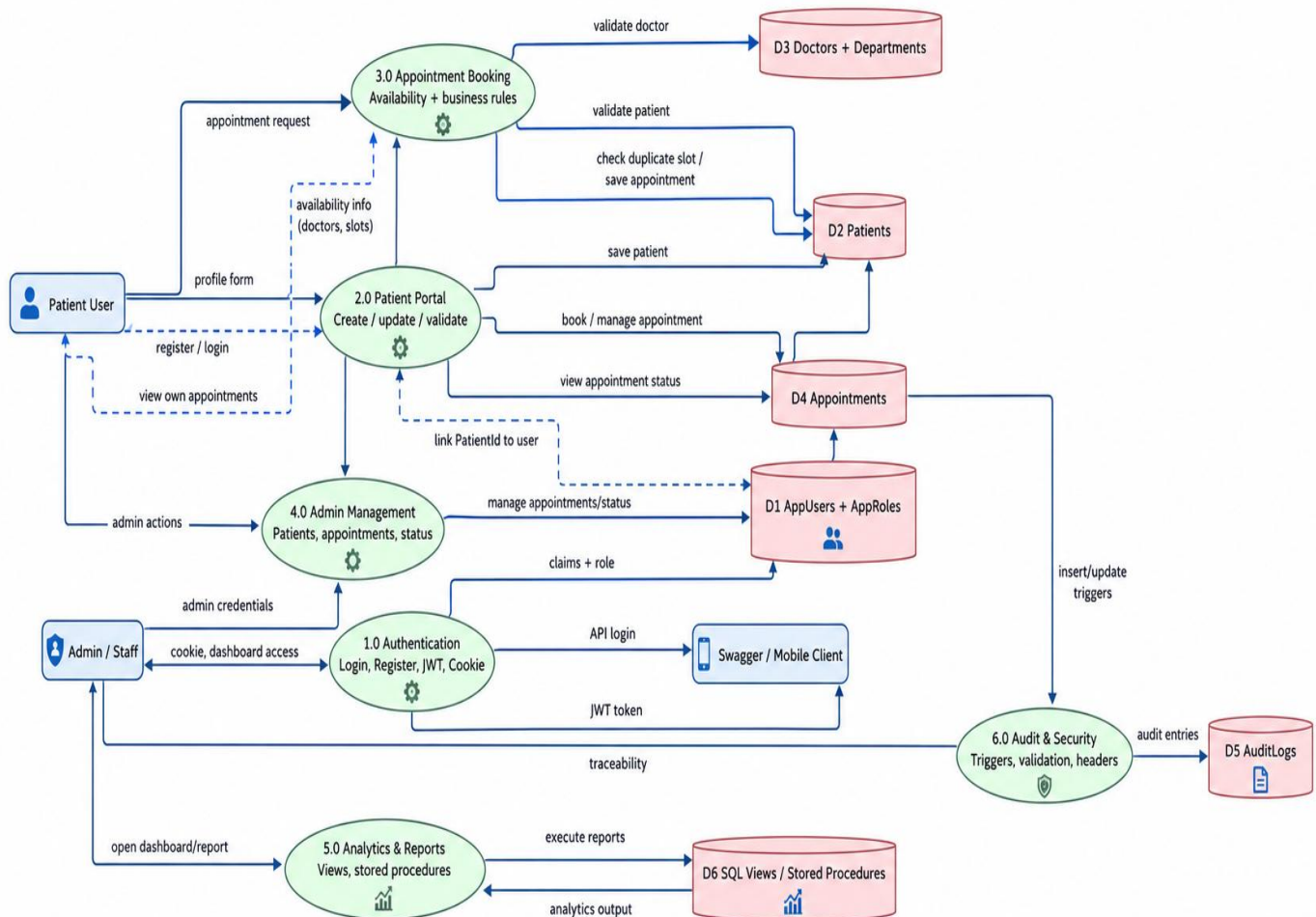


Figure 3: DFD Level 1 - Process Decomposition

The Level 1 DFD decomposes the system into authentication, patient profile management, appointment booking, analytics/reporting, and audit/security processes. Each process interacts with one or more logical data stores.

5. Database Design

5.1 Overview

The database HealthcareCareAnalyticsDB is designed in normalized form to reduce duplication and enforce referential integrity. The core entities are Patients and Appointments, supported by Doctors, Departments, MedicalRecords, AuditLogs, AppUsers, and AppRoles.

5.2 Entity-Relationship Model

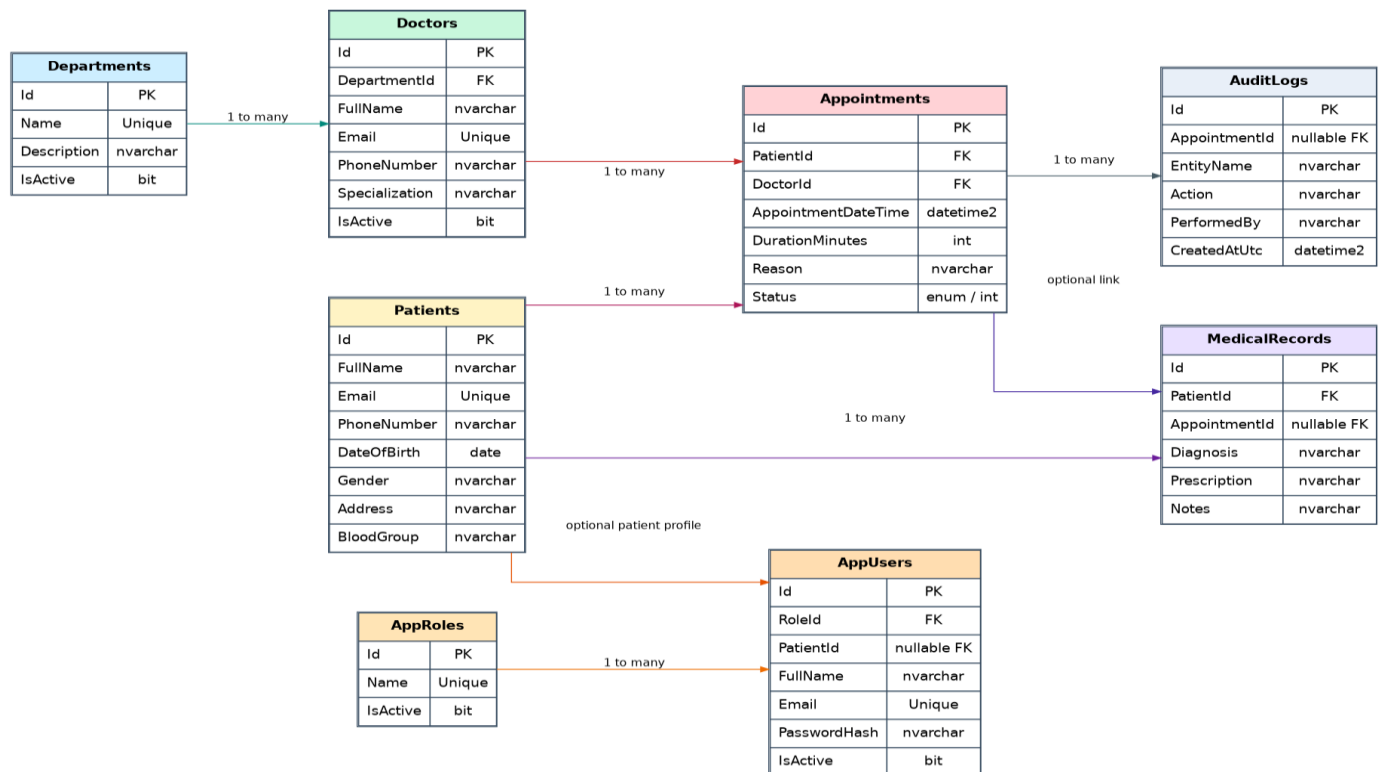


Figure 4: Entity Relationship Diagram - HealthcareCareAnalyticsDB

5.3 Tables and Constraints

Table	Key Columns	Constraints / Purpose
Departments	Id (PK), Name	Name is unique; supports doctor grouping.
Doctors	Id (PK), DepartmentId (FK), Email	Department FK; email uniqueness; active flag.
Patients	Id (PK), Email, PhoneNumber	Email unique; patient demographic and contact details.
Appointments	Id (PK), PatientId (FK), DoctorId (FK), Status	Links patient and doctor; stores schedule, duration, reason, and status.
MedicalRecords	Id (PK), PatientId (FK), AppointmentId nullable FK	Clinical notes linked to patient and optionally appointment.
AuditLogs	Id (PK), AppointmentId nullable FK	Appointment insert/update audit trail through SQL triggers.
AppRoles	Id (PK), Name	Role master table for Admin/User policies.
AppUsers	Id (PK), RoleId (FK), PatientId nullable FK, Email	Login accounts; optional patient profile link for user portal.

5.4 SQL Script Execution Order

1. 01_ddl_schema.sql - creates database objects, tables, keys, constraints, and indexes.
2. 02_dml_seed_data.sql - inserts roles, users, departments, doctors, patients, and sample appointments.
3. 03_views.sql - creates reporting views used by dashboard and analytics.
4. 04_triggers.sql - creates appointment audit triggers.
5. 05_stored_procedures.sql - creates report stored procedures.
6. 06_queries.sql - contains verification and report queries for demo.
7. 07_user_portal_update.sql - adds user portal role/profile linkage changes.

5.5 Normalization Approach

- 1NF: All attributes are atomic; no repeating columns are used.
- 2NF: Non-key attributes depend on the full primary key; tables use identity primary keys.
- 3NF: Patient, Doctor, Role, Department, and Appointment data are separated to prevent transitive dependency.
- Indexes and unique constraints protect data integrity and improve lookup performance.

5.6 Entity Framework Core Usage

- HealthcareDbContext maps entities to SQL Server tables.
- Fluent API configures unique indexes, relationships, delete behaviors, seed data, and trigger-aware appointment mapping.
- Repositories encapsulate data access and Unit of Work coordinates SaveChanges.
- DTO projection prevents leaking EF navigation properties through API responses.
- Async EF methods are used for non-blocking request handling.

6. Layered Project Structure

```

Healthcare Appointment & Care Analytics/
├── src/
│   ├── Healthcare.Core/           # Entities, DTOs, enums, interfaces, domain rules
│   ├── Healthcare.Infrastructure/  # EF Core, repositories, Unit of Work, JWT, hashing, reports
│   ├── Healthcare.API/           # REST API, JWT auth, Swagger, middleware
│   ├── Healthcare.MVC/           # Razor MVC portal, controllers, views, ViewModels
│   └── Healthcare.Tests/         # xUnit tests and fakes
├── database/                     # DDL, DML, views, triggers, stored procedures, queries
├── docs/                         # Report, API docs, security, diagrams, future scope
├── devops/                       # Dockerfile.API, Dockerfile.MVC, compose, GitHub Actions
├── presentation/                 # Final 18-slide PPT
├── run-both-projects.ps1        # Windows demo launcher
└── HealthcareAppointmentCareAnalytics.sln

```

Project	Responsibility
Healthcare.Core	Domain models, DTOs, interfaces, business exceptions, domain service contracts.
Healthcare.Infrastructure	EF Core DbContext, repository implementations, Unit of Work, token service, password hasher, SQL report service.
Healthcare.API	JWT-secured REST endpoints for Auth, Patients, Doctors, Appointments, and Analytics.
Healthcare.MVC	Browser-based portal for admin and patient user workflows.
Healthcare.Tests	Unit tests for business rules and appointment conflict validation.

7. Core Business Rules

Business rules are enforced through MVC/API model validation, domain services, repository checks, EF Core constraints, and SQL Server constraints/triggers.

ID	Rule
BR-1.1	Patient email must be unique.
BR-1.2	Patient phone number must contain valid digits and length.
BR-1.3	New registered accounts receive User role by default; Admin is seed/admin-controlled.
BR-2.1	Appointment must be scheduled in the future.
BR-2.2	Appointment duration must be between 15 and 180 minutes.
BR-2.3	A doctor cannot have overlapping scheduled appointments.
BR-2.4	A patient cannot have overlapping scheduled appointments.
BR-2.5	Completed and cancelled appointments do not block time slots.
BR-2.6	New appointment status is Scheduled by default.
BR-3.1	Admin can manage all patients and appointments; normal users can manage only their own patient profile and appointments.
BR-4.1	Appointment insert/update activity is written to AuditLogs by SQL Server triggers.
BR-5.1	Analytics reports aggregate appointment counts, status counts, and department-wise activity.

8. API Documentation

Base URL in development depends on the local HTTPS port shown by Kestrel, for example <https://localhost:58295>. Protected endpoints require the Authorization header: Bearer <JWT>. Swagger is available at /swagger and the raw OpenAPI JSON is available at /swagger/v1/swagger.json.

8.1 Authentication Endpoint

Endpoint	Purpose	Authorization
POST /api/Auth/login	Authenticates a user and returns a signed JWT with role claims.	Anonymous

Request:

```
{
  "email": "admin@healthcare.local",
  "password": "Admin@123"
}
```

Response 200 OK:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...\"",
  "email": "admin@healthcare.local",
  "fullName": "System Administrator",
  "role": "Admin",
  "expiresInMinutes": 60
}
```

8.2 Patient Endpoints

Endpoint	Description	Access
GET /api/Patients	Returns patient list.	ClinicalStaff policy
GET /api/Patients/{id}	Returns one patient by id.	ClinicalStaff policy
POST /api/Patients	Creates a patient.	AdminOnly policy
PUT /api/Patients/{id}	Updates patient details.	AdminOnly policy

8.3 Doctor Endpoints

Endpoint	Description	Access
GET /api/Doctors	Returns doctors with DoctorId values used for appointment booking.	ClinicalStaff policy
GET /api/Doctors/{id}	Returns one doctor by id.	ClinicalStaff policy

8.4 Appointment Endpoints

Endpoint	Description	Access
GET /api/Appointments	Returns appointment DTOs.	ClinicalStaff policy
GET /api/Appointments/upcoming	Returns upcoming appointments.	ClinicalStaff policy
GET /api/Appointments/{id}	Returns one appointment.	ClinicalStaff policy
POST /api/Appointments	Creates an appointment after validation.	ClinicalStaff policy
PATCH /api/Appointments/{id}/status/{status}	Updates appointment status.	ClinicalStaff policy

POST /api/Appointments

```
{
  "patientId": 1,
  "doctorId": 1,
  "appointmentDateTime": "2026-06-10T11:00:00",
  "durationMinutes": 30,
  "reason": "General health checkup"
}
```

Expected response: 201 Created

8.5 Analytics Endpoints

Endpoint	Description
GET /api/Analytics/summary	Returns dashboard summary counts.
GET /api/Analytics/department-appointments	Returns department-wise appointment report.

8.6 Status Codes Summary

Code	Meaning
200	OK
201	Created
204	No Content
400	Validation / bad request
401	Missing or invalid JWT
403	Authenticated but not authorized
404	Resource not found
500	Unhandled server error

9. Security Implementation

9.1 JWT Authentication



Figure 5: JWT Authentication Flow

The login endpoint validates credentials, loads the user and role, verifies the stored PBKDF2 password hash, and generates a signed JWT. JWT Bearer middleware validates issuer, audience, lifetime, signature, and role claims before protected API controllers execute.

9.2 Role-Based Authorization

- AdminOnly policy protects patient creation and edit operations.
- ClinicalStaff policy protects patient, appointment, doctor, and analytics API endpoints.
- User portal routes are restricted so a registered user can access only their own profile and appointment history.
- Role claims are read only from the authenticated cookie/JWT, never from form body or query string.

9.3 Password Hashing

Plain text passwords are never stored. The Infrastructure layer uses a PBKDF2 password hasher with random salt and secure verification. This supports secure login while keeping the hashing implementation replaceable through IPasswordHasher.

9.4 SQL Injection Protection

- EF Core LINQ is used for CRUD operations instead of string-concatenated SQL.
- ADO.NET report queries use stored procedures and typed readers.
- User input is validated before persistence and never concatenated into raw SQL commands.

9.5 CSRF and Input Validation

- MVC forms use antiforgery tokens and POST actions validate requests.
- DataAnnotations validate required fields, email, date, duration, and phone values.
- Domain services enforce appointment business rules beyond basic UI validation.
- SQL constraints act as a final guard for data integrity.

9.6 Exception Handling and Security Headers

- Exception middleware returns controlled error responses instead of exposing stack traces.
- Security headers include X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Content Security Policy.
- HTTPS redirection is configured for secure transport.

10. Frontend / Backend Connectivity

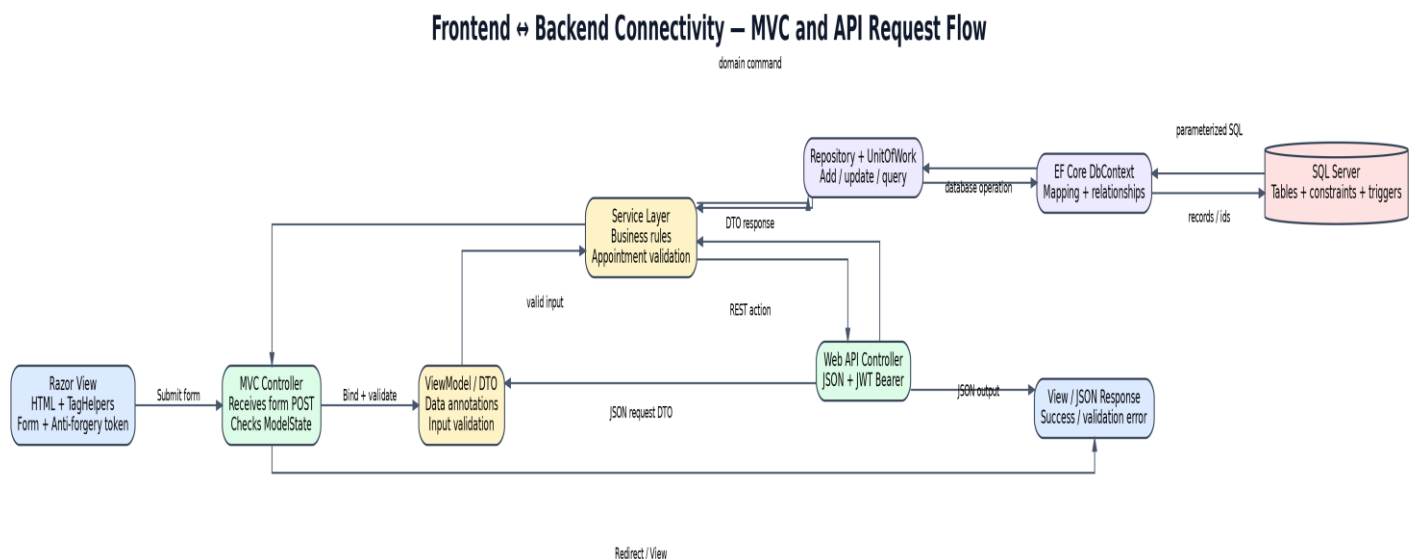


Figure 6: Frontend to Backend Connectivity

The MVC UI uses Razor Tag Helpers and ViewModels. Form data is posted to MVC controllers, then validated with ModelState. Controllers call Core interfaces, Infrastructure repositories save through EF Core, and SQL Server stores the data. AJAX can be used to refresh reports or check appointment-related data without a full page reload.

Razor View -> MVC Controller -> ViewModel Validation -> Service / Domain Rule
-> Repository / Unit of Work -> EF Core / ADO.NET -> SQL Server -> Razor Output

11. Report on Generation Process



Figure 7: Report Generation Flow

- 1) User opens the Reports page in MVC.
- 2) ReportsController calls IAnalyticsReportService.
- 3) SqlAnalyticsReportService opens a SQL Server connection.
- 4) Stored procedure usp_GetDepartmentAppointmentReport executes.
- 5) Result rows are mapped to DTOs.
- 6) Razor view renders the report table; AJAX refresh can update the output dynamically.

12. User Interface Screenshots and Outputs

During final submission, the following screenshots should be captured from the running system and placed in this section. The table defines the exact evidence expected for the evaluator.

Screenshot / Output	What it proves
Login page	MVC authentication screen is available.
Admin dashboard	Admin role reaches dashboard and analytics cards.
Patient list and patient create form	Patient CRUD workflow works with validation.
User registration page	Normal users can register.
Patient portal dashboard	User role redirects to self-service portal.
Create patient profile	User can create their own patient profile.
Book appointment page	User can book appointment using doctor and slot.
My Appointments page	User can see only their own appointments.
Swagger login response	API returns JWT token.
Swagger Doctors / Appointments 200 OK	Protected APIs work with Bearer token.
SSMS Appointments and AuditLogs output	Database insert and trigger audit are verified.

Sample Verified API Output

GET /api/Doctors -> 200 OK

```
[
  {
    "id": 1,
    "fullName": "Dr. Ananya Rao",
```

```

    "email": "ananya.rao@healthcare.local",
    "phoneNumber": "9876543210",
    "specialization": "General Physician",
    "departmentId": 1,
    "isActive": true
  }
]

```

GET /api/Appointments -> 200 OK

Returns appointment DTOs after Swagger authorization with Bearer token.

13. DevOps and AI Modernisation

13.1 Docker Strategy

- Dockerfile.API containerizes the Web API project.
- Dockerfile.MVC containerizes the MVC portal.
- docker-compose.yml can coordinate API, MVC, and SQL Server for conceptual local deployment.
- Environment variables are intended for connection strings and JWT secrets in production.

13.2 GitHub Actions CI/CD

```

GitHub push / pull request
-> dotnet restore
-> dotnet build
-> dotnet test
-> docker build
-> publish artifacts / image (conceptual)

```

13.3 Azure Deployment Concept

- Azure App Service hosts MVC and API containers.
- Azure SQL Database stores production data.
- Azure Key Vault stores secrets such as connection strings and JWT signing key.
- Application Insights can capture logs, metrics, and failures.

13.4 GitHub Copilot Usage

- Generated boilerplate suggestions for controllers, DTOs, and validation comments.
- Suggested SQL report queries and documentation wording.
- All security-sensitive code was manually reviewed before inclusion.

13.5 AI / MCP Agent Concept

A future AI endpoint can summarize appointment-related tickets, patient queries, or care coordination notes. The conceptual endpoint must avoid exposing protected health data outside the approved environment and should include human review before any clinical communication.

14. Testing and Validation

14.1 Unit Testing Approach

The Tests project contains xUnit tests for appointment domain rules. Fake repositories and services are used to isolate business rules without requiring SQL Server.

14.2 Representative Test Cases

Test Case	Expected Result
Build solution with dotnet build	Build succeeds with no compile errors.
Run dotnet test	Appointment domain tests pass.
Login with admin credentials	Admin is redirected to dashboard.
Register new user	User account is created with User role.
Create patient profile	AppUsers.PatientId links to Patients.Id.
Book appointment in future	Appointment is saved and visible in My Appointments.
Book duplicate doctor slot	System rejects overlapping appointment.
Swagger login	JWT token is returned.
Call protected API without token	401 Unauthorized.
Check AuditLogs after appointment insert	Audit trigger creates an entry.

14.3 Validation Strategy

- Client-side validation improves UX but server-side ModelState validation is authoritative.
- Domain service validation enforces appointment overlap rules.
- Database constraints and triggers protect data integrity after application validation.
- Swagger and SSMS are used for end-to-end verification.

15. Final Demonstration

15.1 Pre-Demo Checklist

- SQL Server service is running and connection string points to MSI\RITHINSQLEXPRESS or the active SQL instance.
- Database scripts 01 to 07 have been executed.
- MVC and API projects are set as startup projects or run in separate terminals.
- Default admin login is available: admin@healthcare.local / Admin@123.

- Swagger is opened and JWT authorization is ready.

15.2 Demo Script

1. Introduce the healthcare appointment problem and business objective.
2. Show system architecture, DFD, ER diagram, and database design.
3. Run MVC and login as admin.
4. Show dashboard, patient list, and appointment list.
5. Logout and register a new user.
6. Login as user, create patient profile, and book appointment.
7. Open SSMS and show Patients, AppUsers.PatientId, Appointments, and AuditLogs.
8. Run API, open Swagger, login with admin, copy JWT token, and authorize.
9. Test GET /api/Doctors, GET /api/Appointments, and POST /api/Appointments.
10. Explain security, SOLID principles, design patterns, and future scope.

15.3 Fallback Plan

- If MVC port fails, run `dotnet run --project src/Healthcare.MVC` from terminal and use the URL shown by Kestrel.
- If SQL connection fails, verify SQL Server name in both `appsettings.json` files.
- If Swagger token fails, login again because JWT expires after configured lifetime.
- If appointment trigger fails, confirm trigger-aware EF Core mapping is configured for Appointments.

16. Challenges and Learnings

16.1 Challenges Faced

- Matching EF Core table mapping with existing SQL table names such as AppUsers and AppRoles.
- Handling EF Core SaveChanges with SQL Server triggers on the Appointments table.
- Adding patient user registration without weakening admin-only management features.
- Ensuring protected Swagger endpoints work only after JWT authorization.
- Preventing direct entity serialization issues by returning DTOs from APIs.
- Maintaining a clean layered structure while supporting both MVC and API clients.
-

16.2 Key Learnings

- Controllers should stay thin and delegate work to services/repositories.
- JWT is best suited for API clients while cookies are simpler for server-rendered MVC.
- Database triggers are useful for audit, but EF Core must be configured correctly for triggered tables.
- DTOs keep API responses clean and prevent navigation property serialization problems.
- Strong documentation and diagrams make the project easier to defend during SME Q&A.

17. Future Enhancements

- Doctor schedule calendar and availability management.
- Email/SMS/WhatsApp appointment reminders.
- Online payment and billing module.
- Real-time queue display using SignalR.
- FHIR/HL7 integration for healthcare interoperability.
- Mobile application using the existing REST API.
- Azure Key Vault and Application Insights for production monitoring.
- AI assistant for appointment summary, ticket summarization, and patient query classification.
- Role expansion for Doctor, Nurse, Receptionist, and Lab Technician.
- Advanced analytics with Power BI or dashboard charts.

18. Ideal Scenario and Assumptions

The ideal operating scenario assumes a small-to-medium hospital or clinic where reception staff manage appointments, patients can self-register, and administrators need visibility over scheduling, appointment status, and department-wise care analytics.

Assumption	Reason
Each appointment has one patient and one doctor.	Matches the current normalized appointment model.
A registered user may have one linked patient profile.	Supports patient self-service while keeping admin records centralized.
Doctors belong to departments.	Enables department-wise analytics.
Appointment status changes are audited.	Supports accountability and operational tracking.
API clients use JWT.	Supports mobile/Postman/Swagger clients without server session state.

19. Comparison with Live Healthcare Applications

Feature	This Project	Live Healthcare Application
Patient registration	Available through user portal.	Often includes identity verification, insurance, and consent forms.
Doctor discovery	Doctors listed through portal/API.	Usually includes ratings, availability calendar, language, and location filters.
Appointment booking	Validates future date and slot conflicts.	Adds payment, reminders, teleconsultation, and rescheduling workflows.
Security	JWT, cookies, role policies, hashing, CSRF, validation.	Adds SSO, MFA, audit compliance, encryption at rest, and monitoring.
Analytics	Dashboard and SQL reports.	Advanced BI, forecasting, SLA metrics, and

		operational KPIs.
Interoperability	Conceptual future scope.	FHIR/HL7 integrations with EHR/HIS systems.

20. Conclusion

The Healthcare Appointment & Care Analytics Platform demonstrates a complete .NET capstone solution with layered architecture, secure authentication, role-based authorization, normalized SQL Server design, MVC portal, JWT-secured API, reporting, DevOps readiness, and future AI modernization scope. It is suitable for SME evaluation because it covers backend business logic, MVC UI, database design, REST API security, testing, documentation, and final walkthrough readiness.

The project is academically aligned with the capstone rubric and also follows practical industry direction: separation of concerns, DTOs, repository pattern, Unit of Work, input validation, secure password hashing, SQL injection protection, CSRF protection, audit logging, and container-ready deployment planning.

21. References

- Microsoft Docs - ASP.NET Core: <https://learn.microsoft.com/aspnet/core>
- Microsoft Docs - Entity Framework Core: <https://learn.microsoft.com/ef/core>
- Microsoft Docs - SQL Server: <https://learn.microsoft.com/sql/sql-server>
- Microsoft Docs - Azure App Service: <https://learn.microsoft.com/azure/app-service>
- RFC 7519 - JSON Web Token: <https://datatracker.ietf.org/doc/html/rfc7519>
- OWASP - Authentication, authorization, injection, and CSRF prevention guidance
- Swashbuckle / Swagger for ASP.NET Core API documentation
- Great Learning capstone evaluation requirements and project rubric